L8: Run Time Comparisons

Name	Nature	Comp.	REF
Ravasz	Hierarchical Agglomerative	$O(N^2)$	[11]
Girvan-Newman	Hierarchical Divisive	$O(N^2)$	[9]
Greedy Modularity	Modularity Optimization	$O(N^2)$	[33]
Greedy Modularity (Optimized)	Modularity Optimization	$O(N\log^2 N)$	[35]
Louvain	Modularity Optimization	O(L)	[2]
Infomap	Flow Optimization	O(NlogN)	[44]
Clique Percolation (CFinder)	Overlapping Communities	Exp(N)	[48]
Link Clustering	Hierarchical Agglomerative; Overlapping Communities	$O(N^2)$	[51]

In terms of run-time, the table summarizes the worst-case run time of each of the algorithms we have reviewed so far *(we have not covered Infomap in these lectures)*, including the two algorithms for overlapping community detection *(Cfinder and Link Clustering). N is the number of nodes.* The references refer to the textbook's bibliography.

In sparse networks *(where the number of links is L=O(N))* the Louvain algorithm is the fastest in terms of algorithmic complexity. In dense networks *(where L=O(N^2)),* the Infomap and greedy modularity maximization algorithms are faster.

Remember however that most networks are sparse in practice – and these are only worst-case asymptotic bounds. What happens in practice when we compare the run-time of these algorithms?

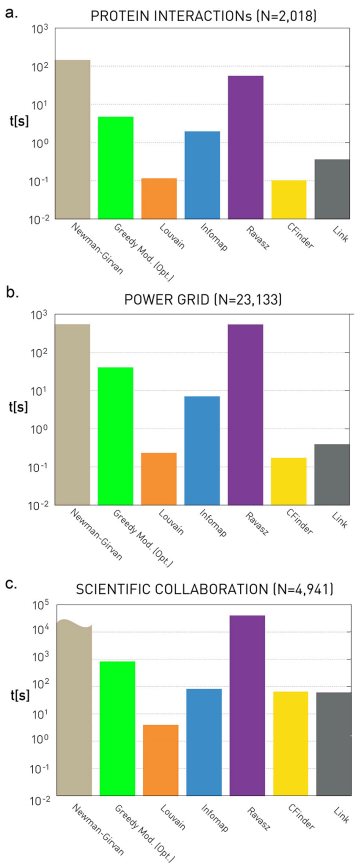

Figure 9.28 from Network Science by Albert-László Barabási

These plots quantify the empirical run-time of the previous community detection algorithms for three small/medium network datasets. The y-axis values are in seconds.

As expected, the fastest algorithm is Louvain. Surprisingly, even though the Cfinder algorithm has an exponential worst-case run-time, it performs quite competitively in practice. The Newman-Girvan algorithm is the slowest among the seven algorithms.

Food for Thought

Make sure that you know how to derive the worst-case run-time complexity of every community detection algorithm we have discussed.